

Cache Invalidation in Caching Services

1. What Is a Caching Service?

A caching service is a centralized cache layer used by backend systems.

Examples:

None

Redis

Memcached

CDN Edge Cache

API Gateway Cache

Unlike browser cache, backend systems **can directly control these caches**.

That means we can:

None

Create entries

Update entries

Delete entries

Expire entries

Evict old entries

2. Why Easier Than Browser Cache

Browser cache problem:

None

Cannot directly clear user browser cache

Need:

None

TTL

Fingerprinting

Revalidation

Caching service:

None

DEL product:44

SET user:7 ...

EXPIRE session:12 300

Direct control available.

3. Common Invalidation Methods

A. Delete Specific Key

Example:

None

DEL product:44

Used when product updated.

B. Update Existing Key

Example:

None

SET product:44 price=799

Used in write-through systems.

C. TTL Expiration

Example:

None

```
SET article:9 EX 600
```

Auto remove after 10 min.

D. Flush Group / Pattern

Example:

None

```
Delete all keys:
```

```
product:*
```

Used after bulk updates.

4. When Cache Is Full

Cache memory is limited.

Need **replacement policy** = choose what to remove.

5. Common Cache Replacement Policies

A. Random Replacement

When full:

None

Remove random item

Advantages:

None

Very simple

Low overhead

Disadvantages:

None

May remove hot data

B. LRU (Least Recently Used)

Remove item not used for longest time.

Example:

None

User viewed product A now

Product B unused for hours

Evict B

Best for workloads with recency patterns.

Most popular strategy.

C. FIFO (First In First Out)

Remove oldest inserted item first.

None

First inserted leaves first

Ignores access frequency.

Simple queue model.

D. LIFO / FILO

Remove newest inserted item first.

None

Last inserted removed first

Less common in caching.

6. Example

Cache contains:

None

A B C D

Need space for E.

FIFO:

None

Remove A

Add E

LRU:

None

Remove least recently accessed

Random:

None

Remove any one item

7. Which Policy Is Best?

Depends on traffic pattern.

Policy	Best Use Case
Random	Ultra-simple systems
LRU	Most web/app workloads
FIFO	Predictable stream workloads
LIFO	Rare niche scenarios

8. Real Redis Example

Redis supports:

None

`allkeys-lru`

`volatile-lru`

`allkeys-random`

`volatile-ttl`

Common production choice:

None

`allkeys-lru`

9. HLD Example

E-commerce system:

None

`Product pages cached in Redis`

`Price update occurs`

`Delete product cache key`

`Next request rebuilds cache`

If memory full:

None

LRU removes cold products

10. Tradeoffs

Good invalidation gives:

None

Fresh data

High hit rate

Low DB load

Fast responses

Bad invalidation causes:

None

Stale results

Memory waste

Hot keys evicted

DB overload

11. Interview Talking Point

If asked cache eviction:

None

Use Redis with TTL + explicit delete-on-write + LRU eviction.

Strong practical answer.

12. One-Line Summary

Caching services allow direct cache invalidation (delete, update, expire entries), and when memory fills up, eviction policies like LRU, FIFO, Random, or LIFO decide which cached data to remove.